CS 24302 : COMPILER DESIGN LABORATORY

Lab Session 7 : Processing declarations in a single scope, symbol table and relevant semantic analysis.

Scheduled : September 24 and 25, 2026

Faculty Team : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas

Research Scholar : Swarna Aishwarya Twinkle

Recap : 1. Generate a parser for C expressions using both ambiguous and unambiguous grammars.
2. Add semantic actions to the grammar to calculate the value of an expression
3. Understand how a lexer can effectively communicate with its parser and perform semantic actions along with parsing.

Objective :

1. Grammar writing for declaration of scalar variables of C.
2. Creation of a simple symbol table structure to save input declarations.
3. Semantic analysis that detects errors in declarations using the symbol table of item 2 above.

Remarks 1: The sizes in bytes of basic data types on your server can be obtained by writing a C program. For example, on my laptop the sizes in bytes of numeric types are as follows.

char	short	int	long	long long	float	double	long double
1	2	4	8	8	4	8	16

You may write a C program to determine the sizes on your server and use them to write your semantic actions.
2. Read the handout to learn how to override the default definition of the variable "yylval" to pass and set attributes of grammar symbols before you start solving the problems on this sheet.

P1. The objective of this problem is to design a declaration parser using the grammar G, with nonterminals {start dlist decl type}, terminals {ID SHORT DOUBLE , ; } and 6 rules given below.

start → dlist dlist → dlist decl ';' | decl ';' decl → type ID | decl ',' ID type → SHORT | DOUBLE

Read the lex and yacc scripts, “declbasic1.l” and “declbasic1.y”, 2 sample input files, "declinp0.txt" and "declinp1.txt".
Read both the scripts carefully and identify the features of lex and yacc used to communicate attributes from the scanner to the parser. Generate a parser with the given resources and test the output of the parser on the given input files.
(a) The output for the 1st input file reports an error. Identify the source of the error, by examining the 2 scripts, from among {lexical, syntax, semantic} and write your answer with justifications.
(b) Examine the output for the 2nd input file and compare it with the output of gcc for the same content. State with justifications for the difference our parser and that of gcc.
(c) Analyse the output of gcc and comment on its precision, with respect to whether all errors are reported, whether the error reports are correct, etc. You may try g++ compiler on the same input and observe its output.

Resources Given : declbasic1.l declbasic1.y declinp0.txt declinp1.txt

P2. The parser of P1, accepts all declarations that are syntactically valid, but is not capable of detecting semantic errors in declarations. An essential requirement for detecting semantic errors in declarations is to save the attributes of all declarations that are both syntactically and semantically valid in some data structure D. When an unknown syntactically valid declaration is parsed, its semantic validity can be checked with those saved in D. In case the new declaration is semantically valid, it is to be saved in D, otherwise issue an error message, skip this declaration, and continue parsing. A hash table is known to be an efficient choice for D, however in our experimentation, we shall use a simple sequential table for ease of implementation. The structure of the symbol table, is drawn in problem P3 below.

Sample C code for a simple symbol table is given in, "symtab-array.c" for illustration. You can un-comment the main() function, compile and test its working on sample input file, "symtab-inp.txt". It is advisable that you design your symbol table and write C code for the same.

Resources : symtab-array.c symtab-inp.txt

P3. Design a declaration parser that uses a symbol table to save all semantically valid declarations and reports errors for declarations that are NOT semantically valid. The semantic error message to be issued is illustrated on the following input, "declinp1.txt" and desired output, "declinp1.txt" . Note that semantic errors are different than that given out by gcc.

Sample input, "declinp1.txt"	**Desired Output,** "declout1.txt"
short a, b, b; short b; double x, a; short a1, b1, a1, x;	Redeclaration Error : b already declared as short Redeclaration Error : b already declared as short Conflicting Redeclaration Error : a earlier as short now as double Redeclaration Error : a1 already declared as short Conflicting Redeclaration Error : x earlier as double now as short Number of declaration statements = 4 Display Contents of Symbol Table Index Name Type Size |Symbols|/offset [0] global symtab 16 4 [1] a short 2 0 [2] b short 2 2 [3] x double 8 4 [4] a1 short 2 12 [5] b1 short 2 14 Space for symbol table: 16 bytes

Resources : declinp1.txt declout1.txt

Bonus Problem Consider grammar G, with nonterminals {start dlist decl type elist}, terminals {ID SHORT DOUBLE , ; } and 8 rules as given below.

start → dlist dlist → dlist decl | decl decl → type elist ';' elist → elist ',' ID | ID

type → SHORT | DOUBLE

The symbol table structure remains the same as that of P3. The difference in P4 is that all identifiers in a single declaration statement are first collected into a list and the list appended to the symbol table in a single call to append(). You would need to define a list data structure for this solution. Outline of the design involves creating a single element list using makelist() and append() to add new elements at the end of the list. Semantic checks have to be added at appropriate places in the actions to ensure that semantic errors are detected and reported. Some nonterminals may have a list data structure associated as its attribute as shown below.

elist → ID {elist.list = makelist (ID.name);}

| $elist_1$ ',' ID {elist.list = append($elist_1$.list, ID.name);}

decl → type elist ';' {insert(symtab, elist.list, type.name, type.width, offset, ….); ...}

End of the Experiment 7